Лабораторна робота №7. Observability: метрики, логи, дашборди й відкат за метриками

Дисципліна: «Технології DevOps» · Студент: MarcoGr

Репозиторій: https://github.com/MarcoGr11/DevOps-gitops

Середовище: Kubernetes v1.37.0 (minikube, kubectl v1.37.1), Argo CD v3.6.1, Argo Rollouts v1.10.1 (чарт 2.43.8), kube-prometheus-stack 92.18.0 (Grafana 13.3.0), Loki 3.7.9 (чарт 20.4.1), Alloy v1.21.3 (чарт 1.15.0), Helm v3.22.0

Застосунок із лаби №6 віддає власні метрики `http_requests_total` і `http_request_duration_seconds` на `/metrics` і пише кожен запит у stdout рядком JSON. kube-prometheus-stack, Loki і Alloy встановлені з Git окремими Application, а ServiceMonitor, дашборд Booking API і AnalysisTemplate лежать у чарті `charts/booking`. Між кроками canary запускається AnalysisRun: кожні 30 с він рахує частку відповідей без 5xx і вимагає щонайменше 0.95. Змінна `FAIL_RATE` дає змогу випустити образ, який відповідає помилкою на задану частку запитів.

1. Стек моніторингу через Argo CD

1. Застосунки стеку й поди monitoring

Перевіряємо, що kube-prometheus-stack, Loki і Alloy встановлені з Git окремими Application, і всі їхні поди працюють.

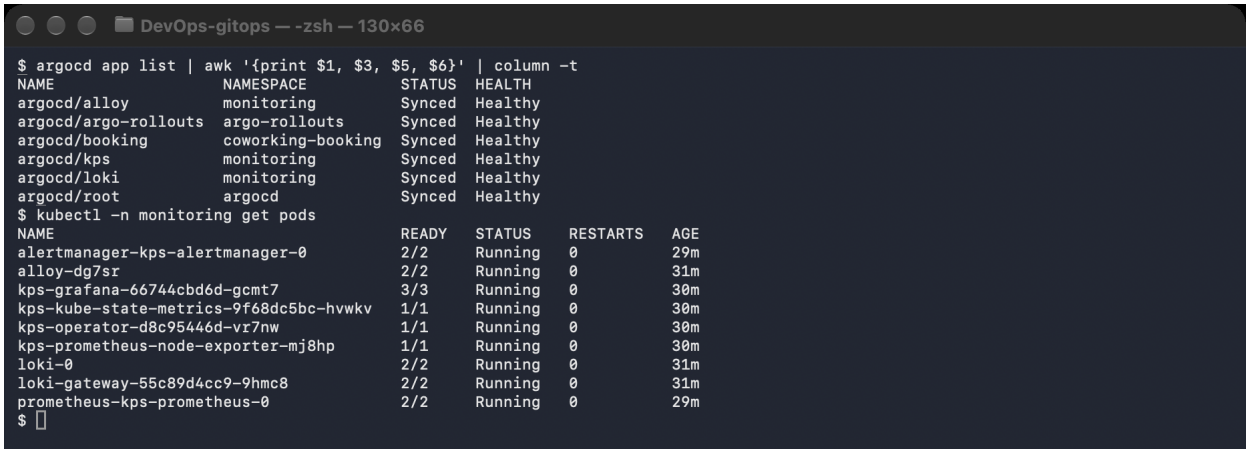


Рис. 1. argocd app list і поди простору monitoring.

Усі шість Application, серед них `kps`, `loki` і `alloy`, мають стан `Synced` і `Healthy`. Поди Prometheus, Grafana, Alertmanager, Loki й Alloy у просторі `monitoring` працюють, перезапусків немає.

2. Метрики застосунку

2. Цілі booking-api у Prometheus

Показуємо, що ServiceMonitor із чарту привів поди api у цілі Prometheus.

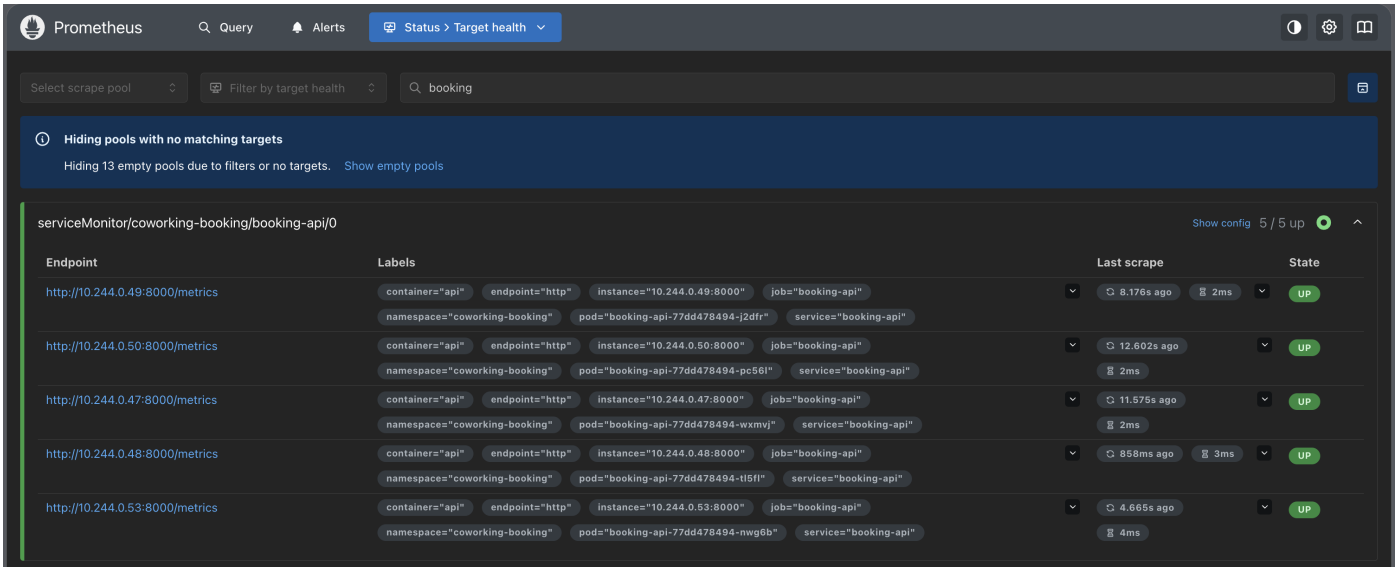


Рис. 2. Prometheus Targets, цілі booking-api у стані UP.

ServiceMonitor `coworking-booking/booking-api` дає 5 цілей із 5 у стані `UP`, по одній на кожен под api. Prometheus знімає метрики з порту 8000 за шляхом `/metrics`, панель браузера обрізано.

3. Запит RPS у Prometheus

Виконуємо PromQL до власної метрики `http_requests_total` під навантаженням генератора `load`.



Рис. 3. Графік RPS застосунку в Prometheus.

Коли запущено генератор `load`, крива росте від 0 і тримається близько 45 запитів на секунду. Просідання майже до 38 припадає на перший поганий реліз, після нього значення повертається.

3. Логи

4. JSON-логи запитів

Показуємо структурований лог запитів, який Alloy збирає в Loki.

```
DevOps-gitops — -zsh — 130×66

$ kubectl -n coworking-booking logs "$(kubectl -n coworking-booking get pods -l app.kubernetes.io/component=api -o name | head -1)
" --tail=6
{"time": "2026-11-04T19:37:26+0000", "level": "info", "msg": "request", "method": "GET", "route": "api/resources/$", "status": 200
, "duration_ms": 6.88, "version": "1.5.0"}
{"time": "2026-11-04T19:37:26+0000", "level": "info", "msg": "request", "method": "GET", "route": "api/resources/$", "status": 200
, "duration_ms": 7.03, "version": "1.5.0"}
{"time": "2026-11-04T19:37:26+0000", "level": "info", "msg": "request", "method": "GET", "route": "api/resources/$", "status": 200
, "duration_ms": 8.27, "version": "1.5.0"}
{"time": "2026-11-04T19:37:26+0000", "level": "info", "msg": "request", "method": "GET", "route": "api/resources/$", "status": 200
, "duration_ms": 7.9, "version": "1.5.0"}
{"time": "2026-11-04T19:37:26+0000", "level": "info", "msg": "request", "method": "GET", "route": "api/resources/$", "status": 200
, "duration_ms": 9.65, "version": "1.5.0"}
{"time": "2026-11-04T19:37:26+0000", "level": "info", "msg": "request", "method": "GET", "route": "api/resources/$", "status": 200
, "duration_ms": 11.09, "version": "1.5.0"}
$
```

Рис. 4. kubectl logs поду api, рядки JSON.

Кожен запит пише один рядок JSON із полями `method`, `route`, `status`, `duration_ms` і `version`. Завдяки цьому Loki може фільтрувати за цими полями без окремих правил розбору.

5. LogQL по 5xx у Grafana Explore

Фільтруємо логи всіх подів api за статусом, розбираючи JSON прямо в запиті.

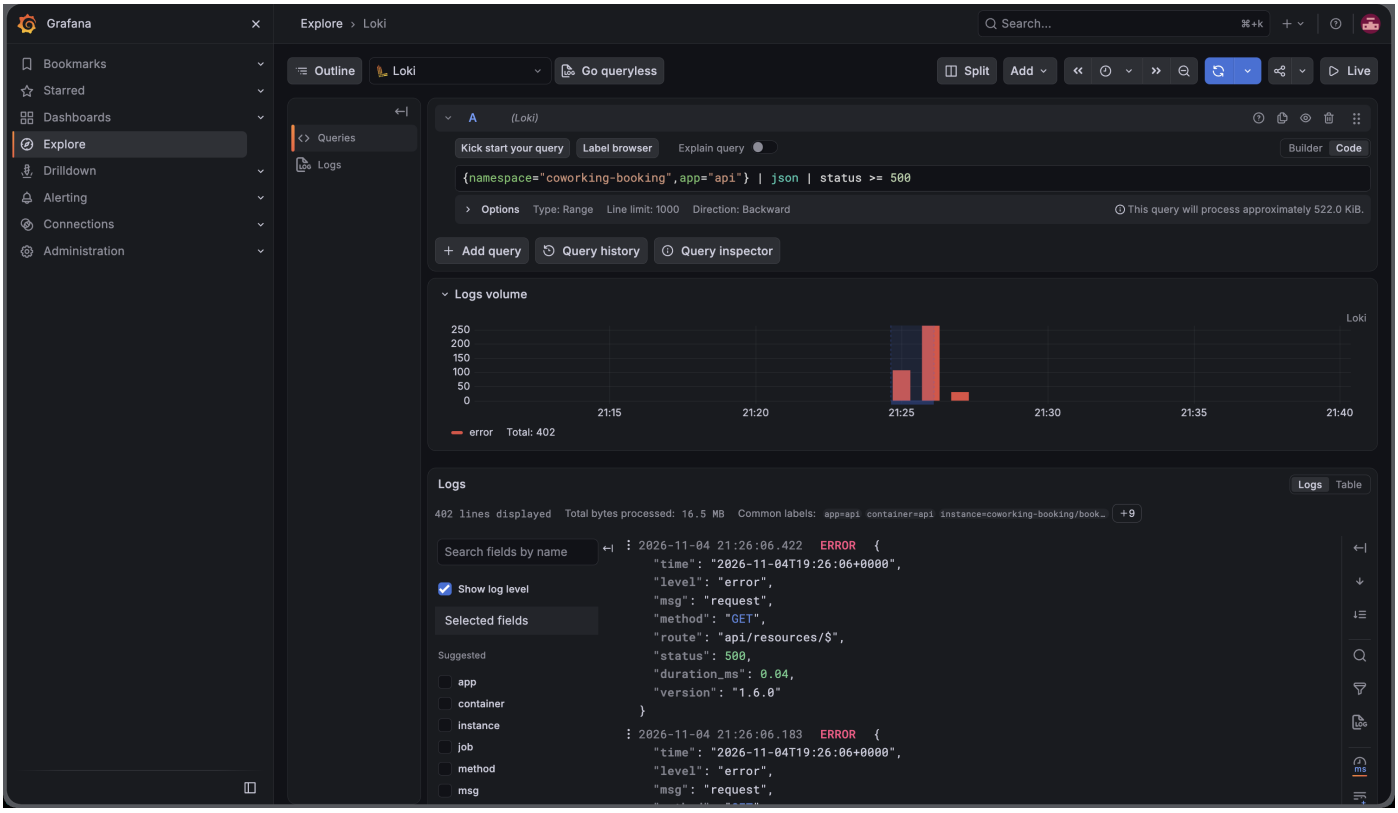


Рис. 5. Grafana Explore, запит за status >= 500.

Запит `| json | status >= 500` знаходить 402 рядки за 30 хвилин, усі рівня error і згруповані в часі першого поганого релізу. Поля рядка видно окремо, зокрема `status: 500` і `version: 1.6.0`.

4. Дашборд

6. Дашборд лежить у Git

Показуємо, що дашборд приходить із Git як ConfigMap із міткою, а не створений вручну в Grafana.

```
DevOps-gitops — -zsh — 130×66

$ kubectl get cm -l grafana_dashboard=1 -A | grep -E "NAMESPACE|booking"
NAMESPACE           NAME                                DATA   AGE
coworking-booking   booking-dashboard                   1      23m
$ git log --oneline -- charts/booking/dashboards/booking.json
2a9e0c4 Add metrics, ServiceMonitor and dashboard
$
```

Рис. 6. ConfigMap дашборду і історія файлу в Git.

ConfigMap `booking-dashboard` із міткою `grafana_dashboard=1` належить простору `coworking-booking`. Файл `charts/booking/dashboards/booking.json` має власну історію в Git, тож дашборд відтворюється з репозиторію.

7. Панель логів Loki на дашборді

Відкриваємо восьму панель дашборду, «Логи помилок (Loki)», на час поганого релізу. На загальному кадрі вона нижче екрана.

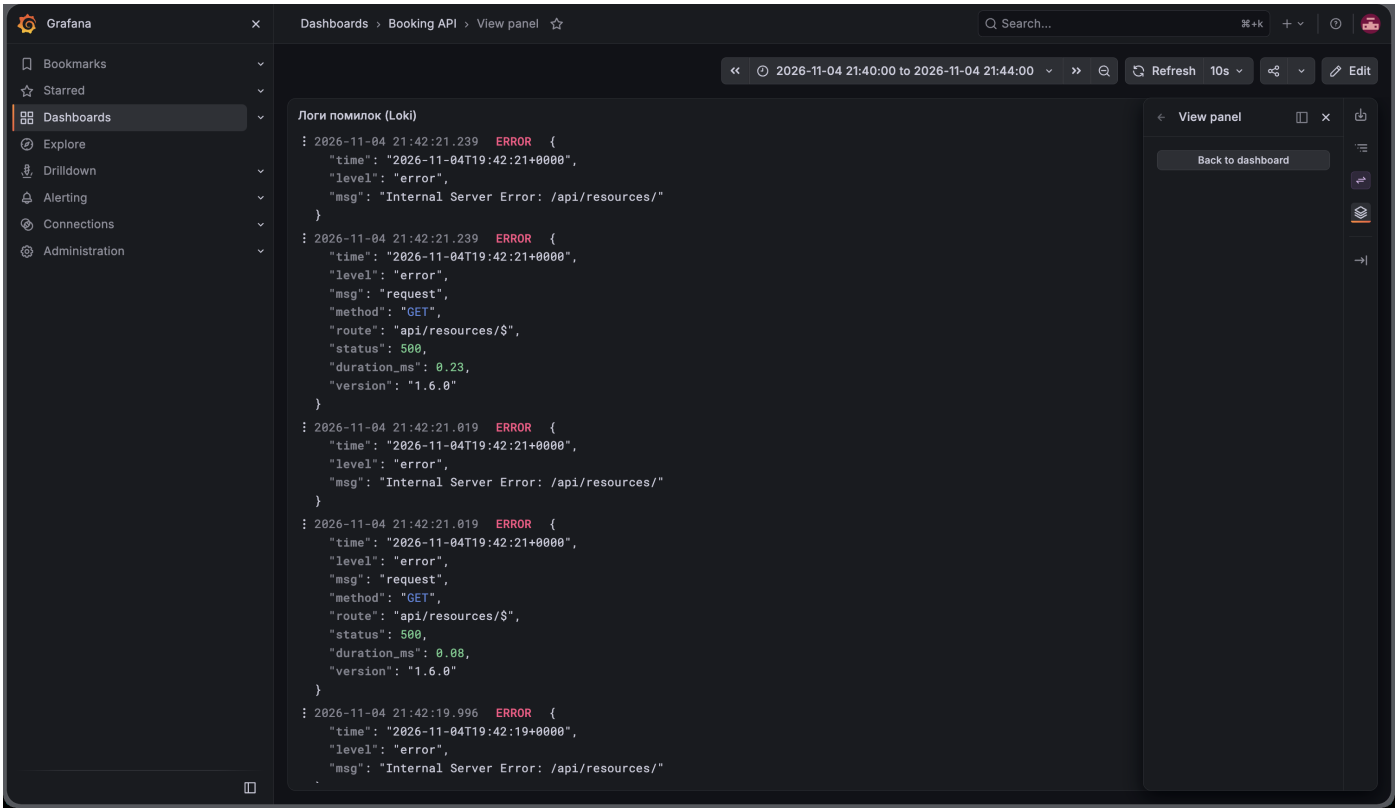


Рис. 7. Панель «Логи помилок (Loki)» дашборду Booking API.

Панель показує рядки рівня error: запит зі `status: 500` і `version: 1.6.0`, а поруч рядок Django `Internal Server Error: /api/resources/`.

5. Навантаження

8. Навантаження і збільшення реплік на дашборді

Показуємо дашборд за час запуску генератора `load` і зміни реплік із 5 до 8 через Git.

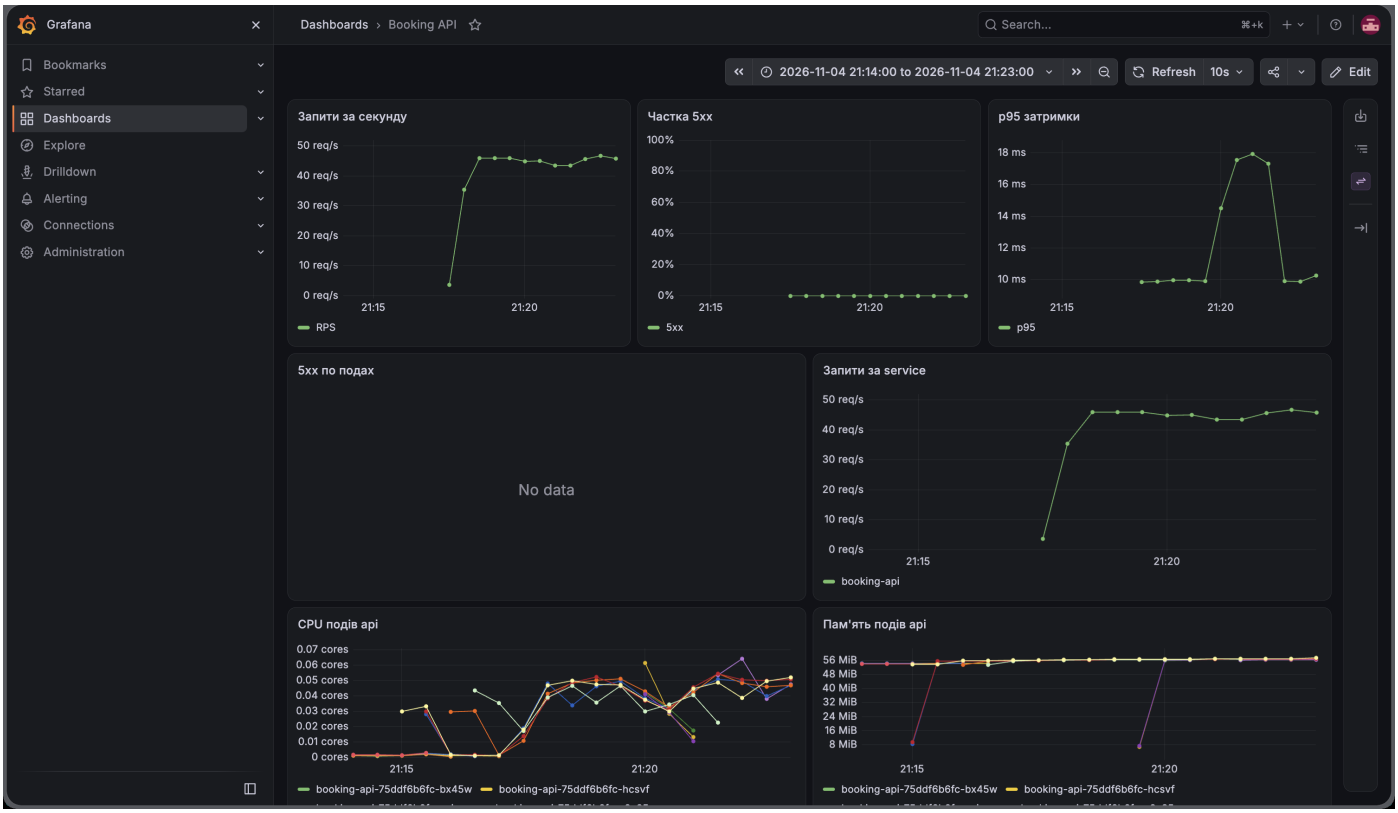


Рис. 8. Дашборд Booking API, 21:14–21:23: старт навантаження і зміна реплік.

Після запуску `load` RPS росте приблизно від 4 до 46 запитів на секунду, а CPU одного пода зростає майже від 0 до близько 0.05 ядра. Затримка p95 тримається близько 10 мс і коротко піднімається до 18 мс близько 21:20, у час зміни реплік; причину цього підйому окремо не досліджували. Prometheus показав 8 цілей `up` о 21:19:57, а після revert знову 5. Помилок немає, панель 5xx порожня.

9. Зміна реплік комітом

Показуємо, що кількість реплік змінено комітом у Git, а не вручну.

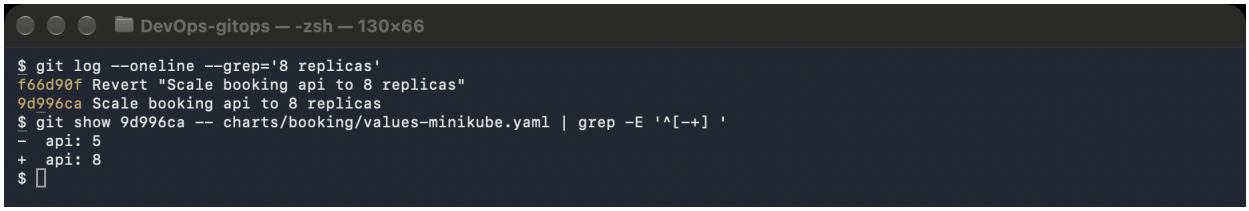


Рис. 9. Коміт зміни реплік і його revert.

Коміт `9d996ca` змінює `api: 5` на `api: 8`, а `f66d90f` його скасовує. Обидві зміни застосував Argo CD, ручного `kubectl scale` не було.

6. Автоматичний відкат за метриками

10. Хороший реліз пройшов аналіз

Показуємо реліз 1.5.0: AnalysisRun успішний, Rollout сам дійшов до кінця без ручного promote.

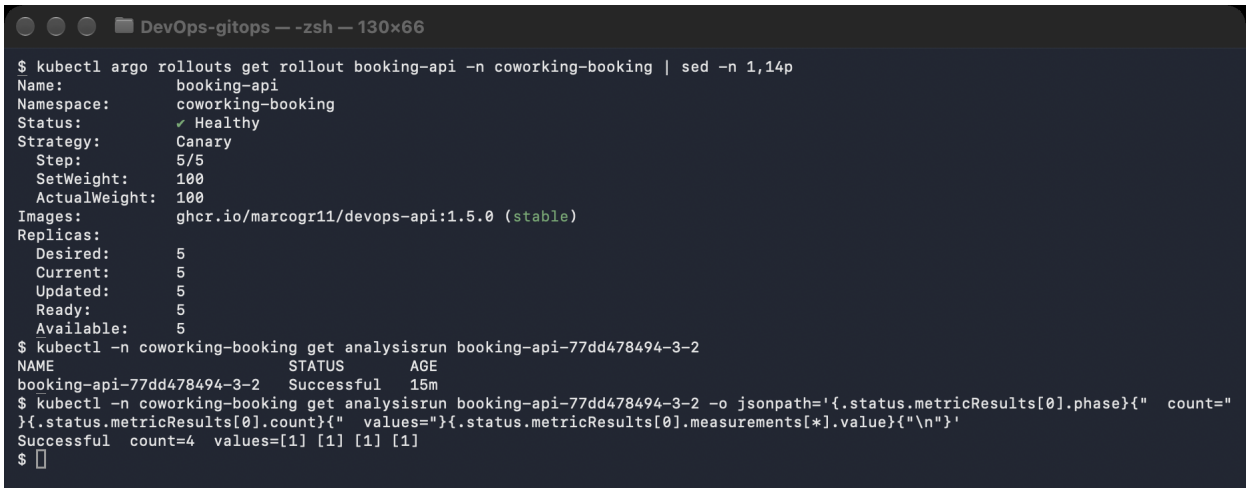


Рис. 10. Rollout Healthy і успішний AnalysisRun.

Rollout `Healthy`, крок 5/5, образ 1.5.0 позначено як stable, 5 подів із 5 готові. AnalysisRun `booking-api-77dd478494-3-2` має стан `Successful`: 4 виміри зі значенням 1, ручного promote не було.

11. Поганий реліз: аналіз провалено

Комітимо реліз 1.6.0 з `FAIL_RATE: "0.5"` і чекаємо: AnalysisRun сам падає, Rollout сам робить abort.

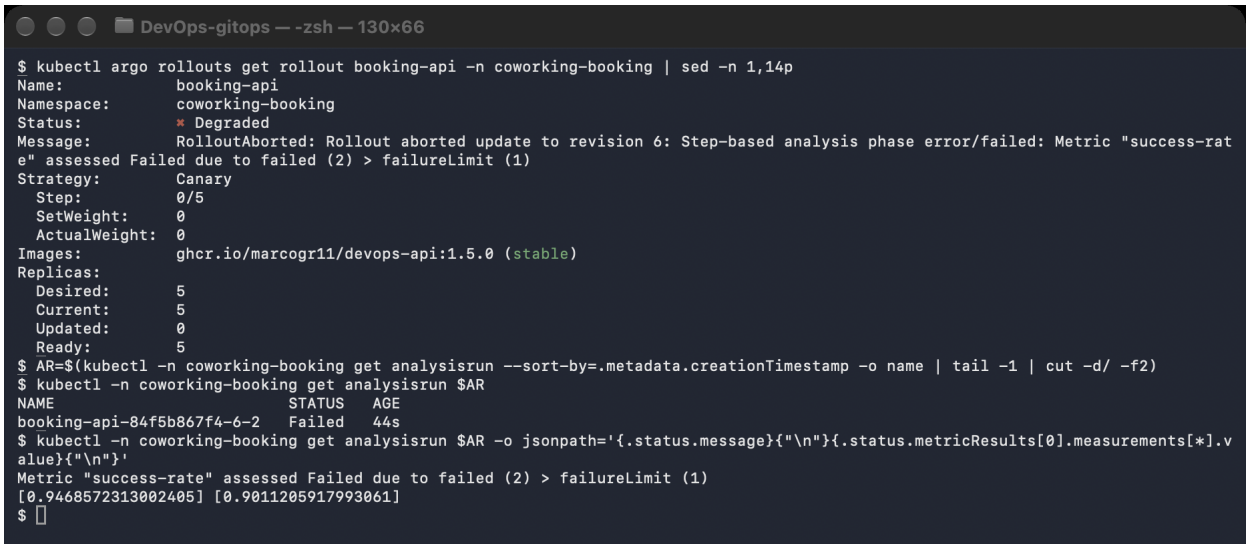


Рис. 11. Rollout Degraded і AnalysisRun Failed.

Аналіз отримав значення 0.9469 і 0.9011, обидва нижчі за поріг 0.95, тому AnalysisRun `Failed` із причиною `failed (2) > failureLimit (1)`. Rollout сам перейшов у `Degraded` з повідомленням `RolloutAborted`, вага 0, stable лишається 1.5.0, усього 76 с від push.

12. Synced і Degraded, користувачі на старій версії

Показуємо, що Argo CD застосував Git (Synced), здоров'я Degraded, а запити обслуговує stable.



Рис. 12. argocd app get booking і цикл curl.

Argo CD показує `Synced to main (ba7626c)` і `Degraded`: Git застосовано, але реліз зупинено. Усі 20 запитів відповіли версією 1.5.0, ще 20 запитів повернули код 200, тож користувачі вже на старій версії.

13. Помилки на дашборді

Дивимось на дашборді 5xx по подах під час поганого релізу.



Рис. 13. Сплеск 5xx лише на canary-поді.

На панелі «5xx по подах» є лише один под `booking-api-84f5b867f4-fdwnn` із приблизно 3.1–3.8 помилки на секунду. Частка 5xx сягнула близько 9% і впала до 0 після abort. Точок лише дві, бо canary прожив недовго.

14. Помилки в Loki лише від нової версії

Той самий запит по 5xx у Loki, тепер видно поле `version`.

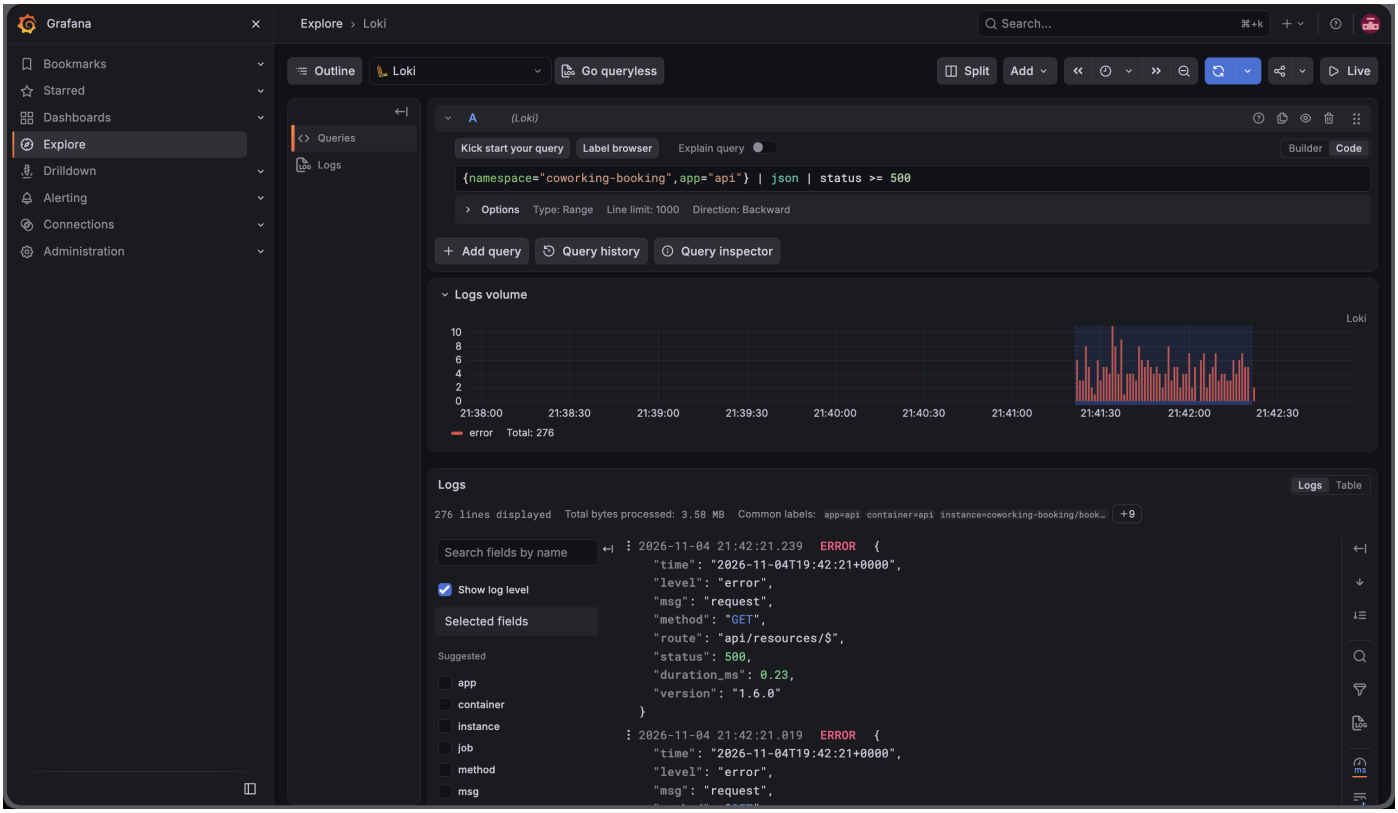


Рис. 14. Grafana Explore, рядки 5xx із version 1.6.0.

За 5 хвилин знайдено 276 рядків зі статусом 500. Окремий запит до Loki підтвердив, що всі вони від одного поду `booking-api-84f5b867f4-fdwnn` і версії 1.6.0.

15. git revert і повернення до Healthy

Abort зупинив трафік, але Git досі каже 1.6.0. Робимо `git revert`, чекаємо Synced і Healthy на 1.5.0.

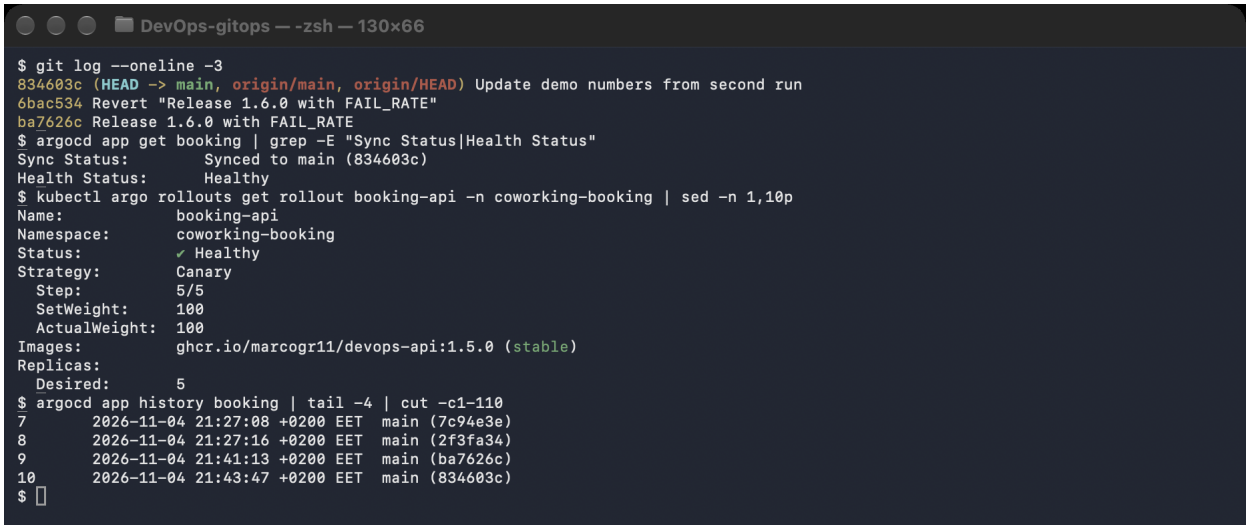


Рис. 15. Rollout Healthy на 1.5.0 і історія Argo CD.

Коміт `6bac534` скасовує `ba7626c`, а наступний коміт змінює лише шпаргалку. Застосунок `Synced` і `Healthy`, Rollout `Healthy` на образі 1.5.0, історія Argo CD містить і поганий реліз (`ba7626c`), і синхронізацію після відкату (`834603c`).

Висновок

У лабі №6 рішення про реліз приймала людина: canary стояв на паузі, доки хтось не виконає promote або abort. Тут те саме рішення приймає AnalysisRun за метрикою, яку застосунок віддає сам: хороший реліз дійшов до кінця без втручання, а поганий зупинився автоматично, і трафік повернувся на stable. Помилки видно на дашборді й у Loki з прив'язкою до поду та версії, тому причину можна знайти без входу в под. Git і далі лишається джерелом правди: після автоматичного abort застосунок має стан Synced і Degraded, доки не зроблено git revert.